

Analyse du marché de l'emploi Data en Île-de-France

Atelier IA — M1 MIAGE, Université Paris 1 Panthéon-Sorbonne Projet en binôme — analyse du marché de l'emploi IT/Data à partir de données collectées automatiquement.

Question fil rouge : *Quelles compétences Data sont les plus demandées en Île-de-France et comment se valorisent-elles salarialement ?*

Sommaire

1. [Plan de récolte de données \(livrable I\)](#)
2. [Stratégie de nettoyage \(livrable II\)](#)
3. [Calcul des indicateurs \(livrable III\)](#)
4. [Analyse \(livrable IV\)](#)
5. [Conclusion \(livrable V\)](#)

1. Plan de récolte de données

1.1 Choix de la source : l'API REST France Travail

Trois familles de techniques de collecte ont été envisagées (vues en cours) : le *parsing* HTML (BeautifulSoup), l'automatisation d'un navigateur (Selenium/Playwright) et l'appel d'une **API REST**. Nous avons retenu l'**API officielle « Offres d'emploi v2 » de France Travail** (ex-Pôle Emploi), selon le principe de priorité : **API si disponible > parsing HTML > navigateur headless**.

Critère	API France Travail	Scraping job board (Indeed, LinkedIn...)
Légalité	Licence de réutilisation officielle	Contraire aux CGU
Robustesse	JSON structuré et versionné	HTML fragile, casse à chaque refonte
Pertinence	Pas de rendu JS à charger	Navigateur headless lourd et lent
Reproductibilité	Requêtes paramétrées rejouables	Dépend du DOM courant

Selenium aurait été le repli si aucune API n'avait existé ; ici l'API domine sur les trois critères, ce qui justifie de ne pas recourir à un navigateur headless.

1.2 Authentification

L'API utilise **OAuth2 avec le grant `client_credentials`** (authentification applicative, sans utilisateur final). Le script `src/01_collecte.py` échange l'identifiant et le secret client contre un jeton *Bearer*, valable pour les appels suivants.

- **Endpoint token** (confirmé sur la doc Swagger) :
`https://entreprise.francetravail.fr/connexion/oauth2/access_token?realm=/partenaire`
- **Scopes** (tous deux obligatoires) : `api_offresemploi_v2 o2dsoffre`

Les identifiants ne sont jamais écrits en clair : ils résident dans un fichier `.env` non versionné (`.gitignore`), et un `.env.example` documente les clés attendues.

1.3 Périmètre de collecte

- **Géographie** : Île-de-France, via le paramètre `region=11`. Ce choix a été préféré au filtre `departement`, **limité à 5 valeurs par l'API** alors que l'IDF compte 8 départements. Le département reste disponible dans chaque offre (champ `lieuTravail`) pour l'analyse fine.
- **Métiers** : mots-clés Data/BI/IA — `data analyst`, `data engineer`, `data scientist`, `business intelligence`, `BI` — un appel (et un fichier brut) par mot-clé pour tracer l'origine de chaque offre.
- **Fenêtre temporelle** : offres créées sur les **30 derniers jours** (`minCreationDate / maxCreationDate`).

1.4 Pagination et respect du quota

L'API renvoie au maximum **150 offres par appel** via le paramètre `range=p-d`, avec les bornes imposées $p \leq 3000$ et $d \leq 3149$ (soit ~3150 offres au plus par requête). Le script parcourt les pages tant que le code HTTP vaut 206 (réponse partielle) et s'arrête sur 200 (page complète) ou 204 (aucune offre). Une pause `time.sleep(0.3)` respecte le quota d'environ **4 appels/seconde** par application. Les appels en erreur (429, 5xx) font l'objet de relances avec back-off.

1.5 Volumétrie obtenue

La collecte a produit **331 offres brutes** (avant dédoublonnage), écrites en JSON UTF-8 dans `data/raw/` (dossier immuable, non versionné). Aucun mot-clé n'a atteint le plafond de l'API : la photographie du marché sur la fenêtre est donc complète.

2. Stratégie de nettoyage

Le script `src/02_nettoyage.py` transforme les JSON bruts en deux tables propres dans `data/processed/`. Principes : `data/raw/` n'est jamais modifié ; tout traitement est rejouable.

2.1 Dédoublonnage

Une même offre peut remonter sur plusieurs mots-clés (ex. un poste « data engineer BI »). Le dédoublonnage par identifiant `id` ramène le corpus de **331 à 299 offres uniques** (–32 doublons).

2.2 Normalisation des champs

Les objets imbriqués sont aplatis : le **département** est dérivé du préfixe « NN - VILLE » de `lieuTravail.libelle` ; l'entreprise, le type de contrat, l'expérience, le métier ROME et le secteur d'activité sont extraits dans des colonnes dédiées. Les dates sont typées.

2.3 Parsing du salaire

Le salaire est un **texte libre** (`salaire.libelle`) présent sur une minorité d'offres. Une expression régulière extrait la période, le minimum et le maximum des formats observés (`Annuel de 50000.0 Euros à 55000.0 Euros sur 12 mois`, `Horaire de 12.31 Euros...`, etc.), puis **normalise en base annuelle** :

- *Annuel* : tel quel ;
- *Mensuel* : $\times$ nombre de versements (12 par défaut) ;
- *Horaire* : $\times$ 1820 h/an (hypothèse temps plein 35 h $\times$ 52 semaines).

Ces hypothèses sont explicitées car elles introduisent une approximation (notamment l'horaire, sensible au temps partiel). **Taux de salaire annuel exploitable : 27 % des offres** — limite assumée et documentée.

2.4 Détection des compétences

Le champ structuré `competences[]` n'est renseigné que sur **~15 %** des offres, ce qui est insuffisant pour répondre à la question fil rouge. Nous avons donc construit un **lexique curé de technologies Data** (langages, bases, big data, cloud, BI, ML, outils) et détecté leur présence par expressions régulières dans l'intitulé, la description et les libellés de compétences. La couverture passe ainsi à **75 % des offres**.

Ce choix méthodologique a un coût : de **rares faux positifs** subsistent sur les motifs courts (ex. `SAS` confondu avec la forme juridique, `R` la lettre isolée), estimés à environ 5 % sur ces motifs après contrôle manuel — un compromis jugé acceptable au regard du gain de couverture. La table `competences_long.csv` (format long `id, competence`) facilite les comptages en aval.

3. Calcul des indicateurs

Le script `src/03_calculs.py` produit les agrégats consommés par le notebook, sans recalcul lourd côté analyse :

- `top_competences.csv` : volume et part des offres par compétence ;
- `competences_salaire.csv` : salaire annuel médian (et quartiles) par compétence, **filtré sur un effectif ≥ 5** pour écarter les médianes peu fiables ;
- **répartitions** par département, type de contrat, expérience et métier ROME ;
- `kpi_global.csv` : indicateurs de synthèse.

La **médiane** (et non la moyenne) est utilisée pour les salaires afin de résister aux valeurs extrêmes (un salaire aberrant à 492 k€ est présent dans les données).

Chiffres clés du corpus : 299 offres · 97 entreprises · 53 compétences distinctes · salaire annuel médian **50 000 €** (Q1 $\approx$ 37 k€, Q3 $\approx$ 56 k€) · 69 % de CDI.

4. Analyse

L'analyse complète et les visualisations figurent dans `notebooks/04_analyse.ipynb` (ré-exécutable de bout en bout). Principaux résultats :

- **Concentration géographique** : Paris (75) et les Hauts-de-Seine (92) concentrent l'essentiel des offres (≈ 109 et 105 offres), loin devant le reste de l'IDF — reflet de la localisation des sièges et ESN.
- **Compétences les plus demandées** : un socle **SQL** ($\approx 40\%$ des offres) et **Python** ($\approx 37\%$), suivis de la **BI/dataviz** (Power BI, Excel, Tableau) et de l'écosystème **cloud / big data** (Azure, GCP, AWS, Spark, Databricks).
- **Valorisation salariale** : les compétences **cloud et big data** (GCP, AWS, Azure, Spark, Kafka, Databricks, Snowflake) affichent les médianes les plus élevées, au-dessus du socle analytique.

Le croisement **demande × valorisation** distingue deux profils : des compétences *socle* très demandées mais peu différenciantes salarialement (SQL, Python), et des compétences *premium* (cloud, traitement distribué) qui tirent les rémunérations vers le haut.

5. Conclusion

5.1 Réponse à la question fil rouge

En Île-de-France, le marché Data repose sur un **socle SQL + Python** quasi universel, complété par la **BI** et, de plus en plus, par **l'ingénierie de la donnée dans le cloud**. Côté rémunération, **ce sont les compétences cloud et big data qui se valorisent le mieux** : SQL et Python sont nécessaires mais peu distinctifs, tandis qu'un profil *data engineer* maîtrisant un cloud (Azure/GCP/AWS) et le traitement distribué (Spark/Databricks) se situe en haut de l'échelle salariale, devant le *data analyst* BI seul.

5.2 Limites et biais

- **Biais de source** : les offres France Travail ne représentent **pas le marché total**. Beaucoup de postes Data cadres sont diffusés via d'autres canaux (LinkedIn, cabinets, cooptation) ; les résultats sous-estiment probablement le haut du marché.
- **Couverture salariale** : seules 27% des offres affichent un salaire exploitable — les médianes par compétence reposent sur de petits effectifs (d'où le seuil ≥ 5), à interpréter comme des ordres de grandeur.
- **Détection lexicale** : quelques faux positifs résiduels ($\sim 5\%$ sur les motifs courts), et le lexique, bien que curé, n'est pas exhaustif.
- **Fenêtre temporelle** : la collecte sur 30 jours est une photographie, non une tendance ; elle peut refléter une saisonnalité.

5.3 Pistes d'amélioration

Élargir la fenêtre temporelle et croiser plusieurs sources (autres API, données INSEE/DARES) ; enrichir le lexique et fiabiliser la détection (NLP) ; suivre l'évolution dans le temps pour passer d'une photographie à une analyse de tendance.

Pour reproduire l'ensemble des résultats, voir le `README.md` (installation et pipeline 01 → 02 → 03 → notebook).